MECC: Single-GPU MoE Inference with Compiler-Guided Three-Tier Expert Caching

***Abstract*—Expert offloading enables single-GPU inference for mixture-of-experts (MoE) models. Repeated HDD reads make expert residency a source of inference latency. We present MECC, the MoE Expert Cache Compiler, for profile-guided placement and joint HBM/CPU caching. Its compiler allocates slots by layer and emits the expert IDs used for initialization. The runtime promotes router-selected experts and retains displaced GPU weights in the host cache. The delivery path orders HDD reads and H2D copies before GPU consumption. On one A100, MECC gives 1.77× decode speedup over streaming on FP16 Mixtral-8x7B. The evaluated Mixtral perplexity is 3.427. The emitted residency plan connects initial expert placement to request-driven promotion and demotion. Joint retention reduces HDD demand by 73.9% relative to independent caches.**

***Index Terms*—Mixture of experts, single-GPU inference, expert offloading, three-tier caching, hardware/software co-design.**

I. Introduction

Mixture-of-experts (MoE) models scale parameter capacity through sparse expert activation [1]–[4]. For example, Mixtral selects two of eight experts in each layer. A single-GPU deployment can use high-bandwidth memory (HBM), host memory, and a hard disk drive (HDD) to hold that pool. The selected expert's location determines its delivery path. HBM residents are ready for GPU execution; host residents require a host-to-device (H2D) copy; cold experts require an HDD read followed by H2D. Layer dependencies expose these reads on the inference critical path. Prefill can reuse fetched weights across prompt rows, and decoding can reuse them across output tokens. On shared servers, a serving task manages its expert allocation alongside other jobs and model instances. We study how to distribute that allocation across layers and retain weights for recurring requests. All MoE layers share the HBM and host allowances. Increasing one layer's allocation leaves fewer slots for others. The value of another slot depends on the demand already covered in that layer. Placement therefore couples the layers' local request distributions to a shared storage allocation.

Expert parallelism places experts across GPUs [5]–[7]. Single-GPU systems combine expert paging and caching [8]–[11] with CPU/GPU execution [12]–[14]. MoE-CORE combines layer-wise allocation, profile initialization, and online replacement with cross-layer prefetching [15]. It also supports substitution for eligible expert misses. SSD-LLaMA retains GPU victims in RAM and pipelines storage delivery alongside lossless compression and CPU/GPU work partitioning [16]. Edge FlashMoE studies cache replacement for storage-backed expert access [17]. Within such a hierarchy, initial placement and later admission affect different requests. A profile determines which weights are available when serving begins. GPU turnover then determines which of those weights survive on the host. Promoting a CPU expert vacates a host position just as an HBM victim loses its GPU position. Using that position for the victim preserves its place in the reuse order. Figure 1 compares this joint rule with independent HBM/CPU caches on an A100. Joint retention gives 4.46× decode speedup over independent replacement and reduces HDD demand by 73.9%. MECC also gives 1.77× speedup over streaming. These comparisons motivate retaining reusable expert weights across both memory tiers throughout serving.

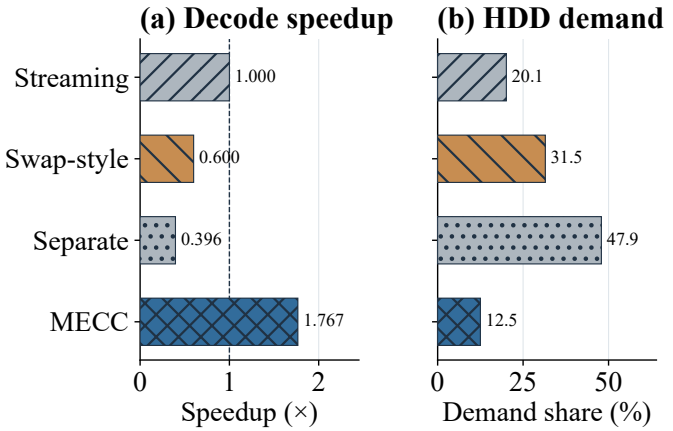


Fig. 1. Mixtral-8x7B on one A100. Decode speedup is relative to streaming; Separate uses independent HBM/CPU caches. Swap-style is the SwapMoE-style adapter. Higher speedup and lower HDD demand are better.

We present MECC, the MoE Expert Cache Compiler, to connect these two stages. Compilation ranks experts within each layer and allocates slots by marginal covered demand. The resulting plan specifies both per-layer capacities and the IDs loaded into HBM and CPU. The runtime maintains a joint recency order across those tiers. A CPU hit promotes the selected expert and demotes the GPU victim; a cold admission can also evict a host resident. A host mirror supplies the weights of each demoted GPU resident. Host admission reuses this copy through a metadata update. Stream tracking keeps tensor storage live through kernel execution. Runtime requests update membership within the compiled capacities. Weight delivery follows the selected experts' read and copy dependencies before GPU execution. Prefill submits cold reads for the active expert batch, while decode loads selected experts on demand. The compiler and runtime address each weight block by its layer/expert ID. This key connects a profiled rank to its initial location, later cache updates, and FP16 execution.

This paper makes three contributions.

(1) **Compiling initial residency.** We emit a model-specific residency plan containing per-layer capacities and ranked expert IDs. A marginal-coverage scan maps each additional slot to the demand it covers.

(2) **Updating cross-tier residency.** We connect the compiled state to joint promotion and victim demotion. Host retention preserves GPU evictions for later reuse, while read/copy ordering prepares selected weights for consumption.

(3) **Evaluating weight delivery.** We compare placement, retention, and scheduling through decode latency, first-token time, source-tier demand, and sample perplexity. Workload and capacity sweeps examine calibration, task transfer, prompt length, batch size, and host allocation. Deployment comparisons report latency and accelerator occupancy.

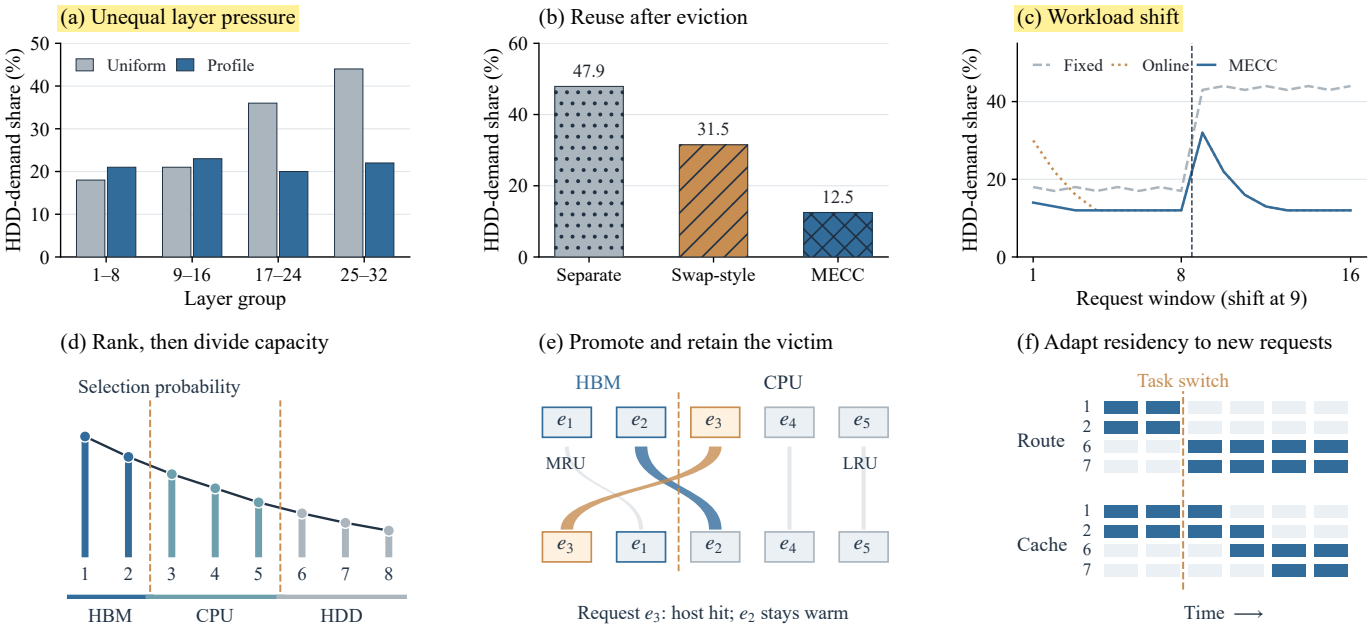


Fig. 2. Expert-cache behavior and MECC's three design choices. (a–c) compare layer pressure, cross-tier reuse, and workload change; (b) counts A100 demand across all phases. (d–f) show layer-specific capacity cuts, joint victim retention, and online membership updates.

II. Background

A. Sparse Execution, Full Weight Capacity

An MoE layer routes each token to a subset of experts and combines their outputs [1], [2], [18]. Mixtral and the DeepSeek family separate active computation from the size of the stored expert pool [3], [4], [19]–[21]. An expert belongs to one layer; equal indices at different layers identify different weights. For input x, let $G(x)$ be the selected set and $g_e(x)$ its normalized gate weights. The layer computes

$$y(x) = \sum_{e \in G(x)} g_e(x) E_e(x). \tag{1}$$

Each gate produces expert requests as a token traverses the model. Decode revisits the layers for each output token. An expert request triggers a residency lookup, weight delivery, and execution on its assigned token rows. The gate weights then combine the expert outputs.

B. Expert Residency on One GPU

Expert parallelism places experts on GPU owners and dispatches token activations to them [5]–[7], [22], [23]. Single-GPU offloading delivers selected weights to the execution device. Experts share HBM with dense weights, activations, and the KV cache [24], [25]. The host cache supplies H2D copies; HDD supplies weights absent from both resident tiers. Heterogeneous inference systems combine these paths with CPU/GPU scheduling [26]–[31]. In prefill, several prompt rows can share a fetched expert. Decode advances one token per sequence, exposing fewer rows to each active expert. The amount of ready computation therefore changes between the two phases. Other systems change transfer size through precision selection [32], [33], expert segmentation [34], [35], or staging and compression [36], [37].

Calibration counts determine expert ranking; runtime accesses determine recency. Within one layer, sequences $[a, a, b, b]$ and $[b, b, a, a]$ have equal counts and different last-access orders. MECC initializes from the ranked profile and replaces residents by recency.

III. Motivation and Insights

A. Allocating Slots by Marginal Demand

Expert demand varies across layers. Some layers concentrate requests in a few ranks; others spread requests across more experts. Figure 2(a,d) illustrates the resulting allocation choice. Its profile-guided plan shifts slots toward the layer groups with more cold demand. The compiler makes this choice by comparing each layer's next uncovered rank. Allocating a slot exposes that layer's following rank as a new candidate. This process distributes a shared allowance across layers and retains the IDs that fill each allocation. Routing locality across models has also been studied independently [38].

B. Preserving Reuse after GPU Eviction

A GPU victim is also a candidate for host admission. Figure 2(b,e) shows a CPU promotion vacating a host position as the GPU selects its victim. Joint retention admits that victim into the vacated position, carrying recency across the tier boundary. Starting from the same allocation and expert order, joint retention reduces HDD demand from 47.9% to 12.5%. A demoted expert enters CPU MRU and remains available until subsequent admissions evict it from CPU LRU. A later selection promotes this host copy through H2D, turning the earlier GPU eviction into a host-cache hit.

C. Updating Residency beyond Calibration

Calibration and serving inputs can select different expert sets. In Figure 2(c,f), a task switch moves requests toward a new expert set. MECC initializes the resident set from calibration and then admits experts requested during serving. Each admission updates recency and brings the cache closer to the active task. Repeated requests keep the new experts near MRU; their unused predecessors advance toward eviction. The profile supplies an initial hot set, and request-driven replacement tracks its evolution. The compiled capacities assign each layer the space used for this adaptation.

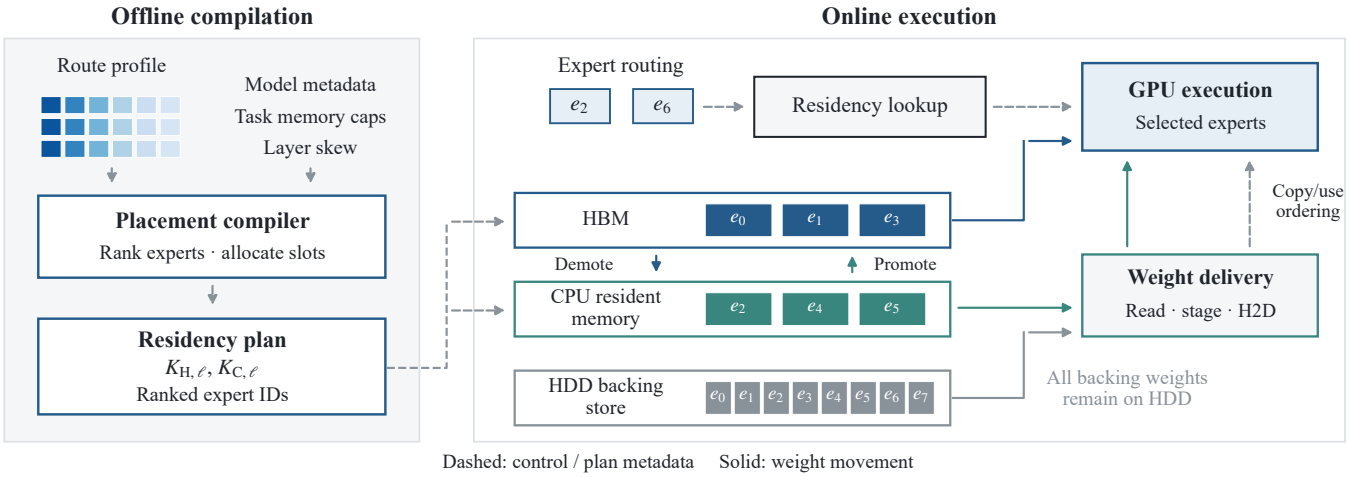


Fig. 3. MECC framework. The placement compiler emits per-layer capacities and ranked IDs. Online execution maps routed expert requests to HBM, CPU resident memory, or HDD, then supplies their weights to GPU computation. Dashed arrows carry control or plan metadata; solid arrows move weights.

IV. Overview

Figure 3 follows the plan from profiling to GPU execution. The compiler receives layer/expert counts, skew parameters, model metadata, and expert allowances. It emits capacities and ranked IDs for each layer. Initialization loads the corresponding HBM and CPU residents; the complete expert pool remains on HDD. The runtime subsequently consumes this plan as an initial cache state.

Consider the selected experts e_2 and e_6 in the framework. The residency lookup finds e_2 on the host and e_6 in the backing store. Promoting e_2 frees a host position for an HBM victim. Loading e_6 then admits another GPU resident and updates the same tier order. The delivery path resolves each weight's read and copy before its consumer executes. The model combines the expert outputs using the gate weights and continues to the next layer.

Residency persists across tokens. Each layer updates its own membership within the compiled capacities. Per-layer recency carries request history from one token to the next. Promotions refresh the active experts, and demotions extend their reuse through host residency. Section V specifies these transitions; Section VI examines the resulting delivery cost.

V. Method

A. Compiling Capacities and Initial Identities

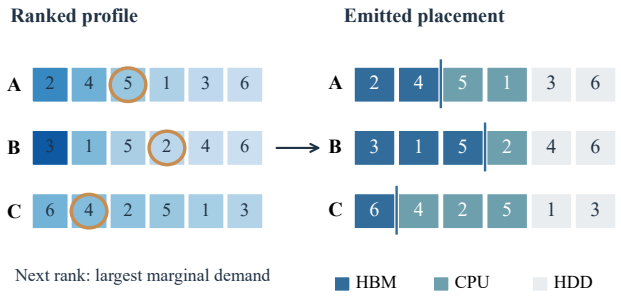


Fig. 4. Compiling the initial state. The left grid contains ranked per-layer demand; circles mark the next uncovered candidates. Shared HBM/CPU slot allowances produce different cuts in the emitted rows. A, B, and C identify different layers.

Consider L MoE layers with N_ℓ experts per layer. For the routed pools considered here, experts have equal size s and layers use the same top-k.

Expert allowances B_H and B_C provide $S_H = \lfloor B_H/s \rfloor$ HBM slots and $S_C = \lfloor B_C/s \rfloor$ CPU slots.

Calibration orders experts by selection counts. A log-count versus log-rank regression supplies α_ℓ as the negative slope over positive counts. The rank model smooths this profile for allocation scoring:

$$q_{\ell,r} = \frac{r^{-\alpha_\ell}}{\sum_{j=1}^{N_\ell} j^{-\alpha_\ell}}. \tag{2}$$

Here $q_{\ell,r}$ is conditional selection mass within layer ℓ, with unit sum. Equal top-k makes this mass comparable across layers. The expert IDs retain the count-derived order. E1 uses uniform allocation; E2 varies the profile and allocation inputs.

Figure 4 shows the HBM and CPU cuts in those lists. Capacities satisfy $\sum_\ell K_{H,\ell} \le S_H$, $\sum_\ell K_{C,\ell} \le S_C$, and $K_{H,\ell} + K_{C,\ell} \le N_\ell$. The adaptive scan reserves one slot per layer in each fast tier, then distributes the remaining slots by marginal coverage.

The scan first assigns HBM slots. At each step, it chooses the layer with the largest next-rank score:

$$\Delta_\ell = q_{\ell,K_{H,\ell}+1}. \tag{3}$$

CPU allocation continues from the first ranks beyond each HBM cut. Every allocation exposes a new next-rank candidate in that layer. Layer ID breaks score ties, and expert ID breaks count ties. The direct scan takes $O(L(S_H + S_C))$ time after sorting. Its score is the additional within-layer selection mass covered by the next slot.

The compiler serializes a residency plan: model dimensions, expert size, capacity arrays, and ranked layer-local IDs. Initialization loads each HBM prefix and the following CPU prefix. The JSON plan is the deployment artifact consumed by the loader. Admission subsequently updates membership within its capacities.

B. Retaining Victims across the Tier Boundary

Each layer maintains two ordered sets, from most-recently used (MRU) to least-recently used (LRU). Their concatenation places HBM residents before CPU residents. An HBM hit moves its expert to HBM MRU. A CPU hit removes its host entry and promotes it to HBM MRU. The GPU LRU victim then becomes CPU MRU. A cold admission follows the same GPU insertion and can remove the CPU LRU. HDD retains the backing weights for every expert.

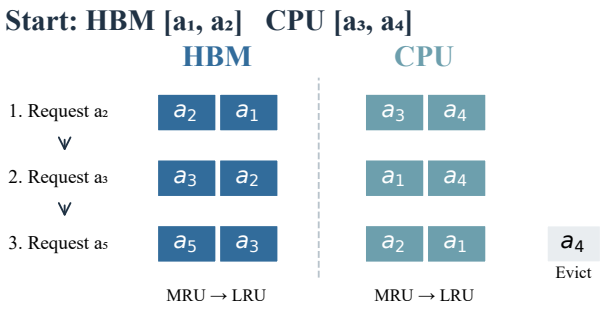


Fig. 5. A continuous request sequence, read from top to bottom. Each row starts from the preceding row's state. Request a_2 refreshes HBM recency; a_3 promotes from CPU; cold a_5 demotes a_2 and evicts a_4. All lists run MRU to LRU.

Figure 5 starts with HBM $[a_1, a_2]$ and CPU $[a_3, a_4]$. Request a_2 gives HBM $[a_2, a_1]$. Request a_3 then promotes a_3 and demotes a_1, producing HBM $[a_3, a_2]$ and CPU $[a_1, a_4]$. Finally, a_5 enters HBM, a_2 becomes CPU MRU, and a_4 leaves CPU LRU. The final lists are $[a_5, a_3]$ and $[a_2, a_1]$.

The resulting order extends from HBM MRU through CPU LRU. Requests enter at its front, and capacity pressure removes experts at its tail. During batch preparation, the loader can prepare several active experts. Temporary references hold their weights until each expert processes its token rows.

Each GPU resident carries a host mirror. Demotion admits this mirror to the CPU cache through a metadata update, saving a device-to-host copy. Tracking the consumer stream keeps tensor storage live until kernel execution completes.

C. Ordering Delivery before Expert Consumption

Prefill groups token rows by selected expert and submits the corresponding cold reads. Each read is resolved when its expert reaches consumption. Figure 6 shows resident work overlapping a pending read for another selected expert. Decode delivers each requested expert on demand. The expert kernel processes its assigned rows, and the gate weights combine the resulting outputs.

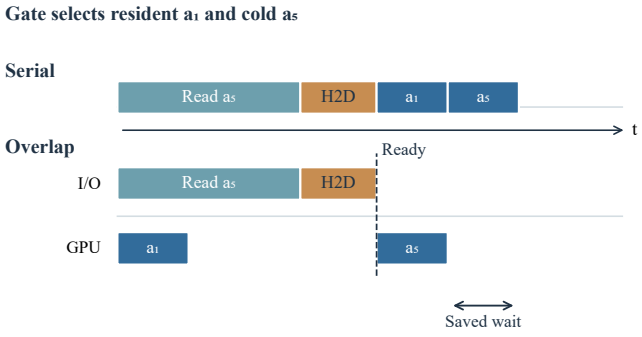


Fig. 6. Prefill delivery for resident a_1 and cold a_5. Computing a_1 overlaps the pending read for a_5. H2D completion releases a_5 for GPU execution; the shortened schedule saves exposed waiting time.

A read future returns the host tensors that supply H2D. The copy stream moves them to the GPU and synchronizes before the loader yields the device tensors. The expert kernel then consumes those tensors. Recording its stream extends their lifetime through kernel completion. This read–copy–compute chain connects storage delivery to the expert's execution point.

Prefetchers use profiles or lookahead to anticipate expert demand [39]–[41]. Batch and request schedulers expose work from concurrent inputs [42], [43]; distributed edge schemes add network delivery [44]. MECC schedules the expert work already selected at the current gate. Retention removes repeated reads and copies. Overlap shortens the exposed part of the remaining delivery.

VI. Evaluation

A. Platform, Protocols, and Metrics

The testbed has one A100 80 GB PCIe and a local HDD-backed RAID volume. FP16 Mixtral experts occupy 336 MiB each. Table I defines the input and residency settings used throughout the evaluation. E1 uses uniform initialization; E2/E3 use calibrated initialization. E4 changes the host-slot allowance.

TABLE I
Execution protocols. Input/output are tokens per prompt. H/C are primary HBM/CPU allowances in GiB. E1/E4 use one run per point.

ID	Input group	Prompts; tokens	H/C
E1	WikiText-2	8; 64/64	50/40
E2	GSM8K	8; 128/128	per model[a]
E3	GSM8K, HumanEval	8; 128/128	50/{40, 64}[a]
E4	WikiText-2	4; 64/16	50/30 or 60
E5	GSM8K	8; 128/128	30/20

[a] Model-specific settings and figure-to-protocol mappings appear in Appendix A.

TPOT measures decode time per output token; TTFT measures time to the first token. Phase speedup divides reference latency by policy latency, using full-precision run values. HDD-demand share is $n_D/(n_H + n_C + n_D + n_{\text{CPUexec}})$. The main access counters aggregate warmup, prefill, decode, and PPL. Their denominator counts expert accesses, including batch-grouped accesses during prefill.

B. Single-GPU Performance

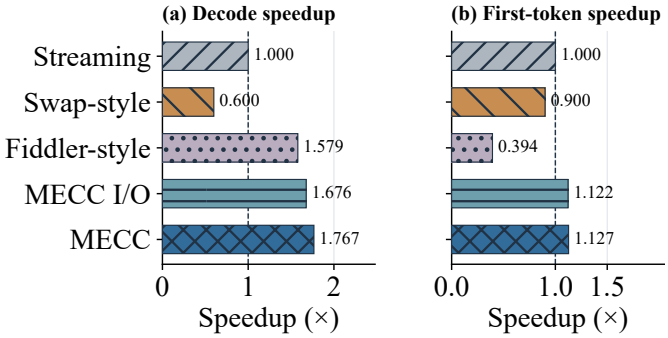


Fig. 7. Decode and first-token speedup on one A100, relative to streaming (1.0). Swap-style and Fiddler-style denote the corresponding mechanism adapters. MECC I/O varies the weight-delivery path. Higher is better.

E1 gives 1.77× decode and 1.13× first-token speedup over streaming in Figure 7. Relative to Fiddler-style CPU computation, these ratios are 1.12× and 2.86×. The phase-specific results distinguish first-token work from recurring expert delivery during generation.

E1 classifies 77.5% of MECC accesses as HBM hits, 10.0% as CPU hits, and 12.5% as HDD accesses. Joint retention gives 2.95× decode speedup over the SwapMoE-style adapter and 4.46× over independent replacement. The independent-cache HDD share is 47.9%. Figure 1 uses these aggregate counters.

On two short correctness inputs, MECC and streaming produce equal expert selections and greedy tokens. Maximum absolute logits and gate errors are zero across three output tokens per input. Appendix A identifies the fixture and shared dispatch path.

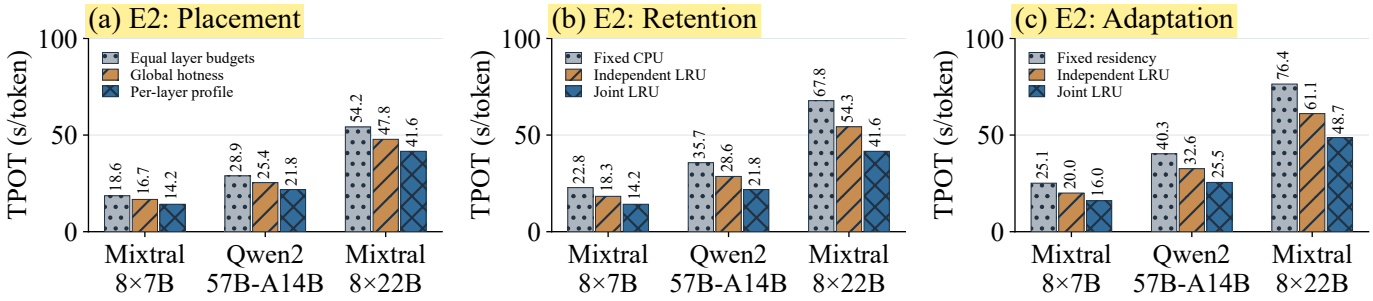


Fig. 8. E2 allocation, retention, and adaptation across three models. Runtime policy is fixed in (a); initialization is fixed in (b,c). Bars report TPOT.

C. Mechanism Attribution

E2 uses smaller resident allowances than E1 for Mixtral-8x7B. Figure 8(a) compares equal budgets, global hotness, and layer profiles. Their TPOT values are 18.6, 16.7, and 14.2 s/token. Panel (b) compares fixed CPU, independent LRU, and joint LRU at 22.8, 18.3, and 14.2. Panel (c) follows a task switch; joint admission changes post-switch TPOT from 25.1 to 16.0.

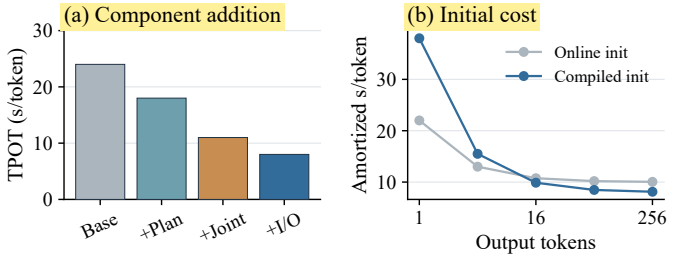


Fig. 9. E3 ablation and initialization amortization. (a) Components under the GSM8K protocol. (b) Online initialization versus compiled initialization; curves include initial placement and decode cost.

For E3 Mixtral-8x7B/GSM8K, adding placement, joint retention, and delivery scheduling gives the TPOT sequence 24, 18, 11, and 8 s/token in Figure 9(a). The successive reductions are 6, 7, and 3 s/token. Joint retention provides the largest step by keeping GPU victims available on the host. Scheduling then shortens the delivery path at 20 percent HDD demand. Panel (b) includes the initial placement cost, connecting this decode gain to the response length.

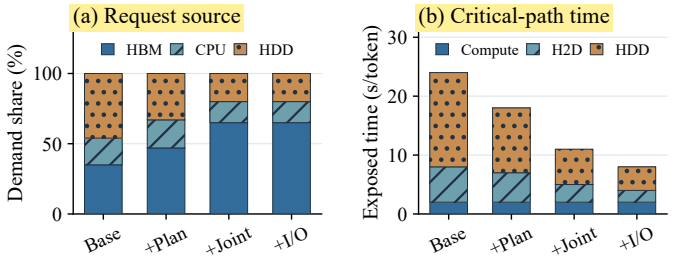


Fig. 10. E3 request sources and exposed delivery time. The final scheduling step changes exposed time at a fixed source distribution.

Figure 10 resolves the E3 component costs. HDD demand falls from 46 to 20 percent. The final scheduling step keeps the 65/15/20 percent HBM/CPU/HDD mix and changes TPOT from 11 to 8 s/token. Exposed HDD time falls by 2 seconds and H2D by 1 second. Retention removes transfers; scheduling shortens their exposed duration.

D. Model and Deployment Scaling

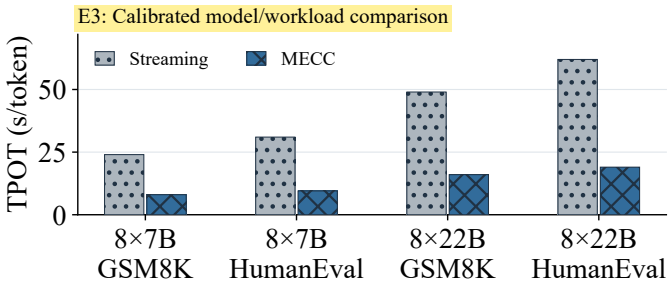


Fig. 11. E3 model/workload comparison. GSM8K and HumanEval use 128 input and 128 output tokens, calibrated initialization, and the model-specific allowances in Table I.

Under E3 calibrated initialization, Figure 11 gives streaming-to-MECC ratios of 3.00–3.26× across the four model/workload groups. The absolute TPOT difference grows from 16 seconds on 8x7B/GSM8K to 43 seconds on 8x22B/HumanEval. Moving from 8x7B to 8x22B doubles MECC TPOT; the corresponding streaming increase is 2.00–2.04×. E3 measures calibrated serving on these tasks. E1 evaluates uniform initialization on WikiText-2; its 1.77× ratio belongs to that configuration.

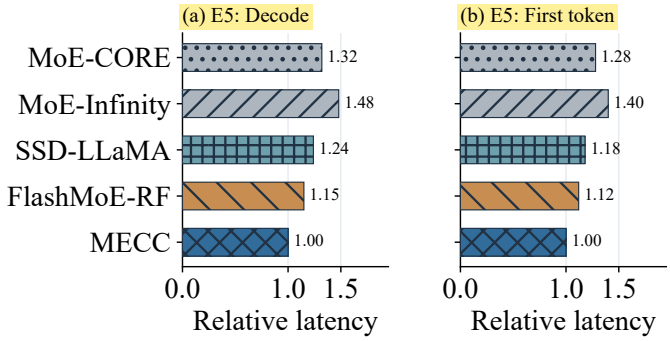


Fig. 12. E5 matched FP16 policy paths. Latency is normalized to MECC within each phase. Lower values are better.

E5 compares profile-based placement, victim retention, and history-based caching through the paths in Figure 12. All paths use routed experts in FP16. Substitution and compression are disabled in this protocol. The comparison asks how the retention decision changes under a shared expert-store format and allowance. MoE-CORE, MoE-Infinity, and SSD-LLaMA supply the corresponding policy designs [10], [15], [16]. Appendix A records each path and the full E1 adapter results.

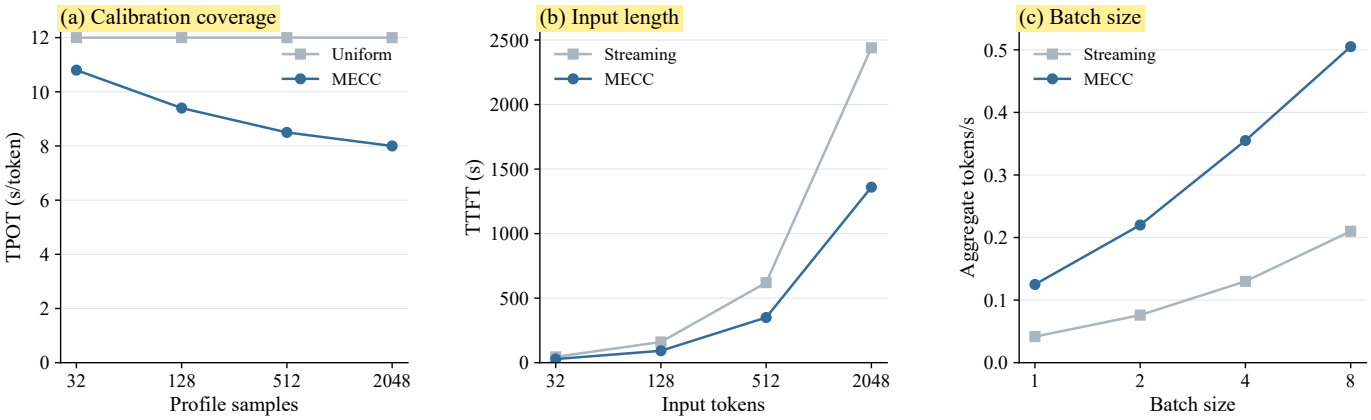


Fig. 13. Profile, input-length, and batch sensitivity. Profile size changes initialization quality, input length changes first-token work, and batch size changes aggregate throughput. The panels vary one workload parameter at a time.

E. Capacity and Workload Sensitivity

E4 gives 1.42× and 5.17× speedup at 30 and 60 GiB CPU-slot allowances. Its four-prompt, 16-output-token protocol differs from E1. At 60 GiB, primary residency is 42 GiB in HBM plus 42 GiB on CPU: all 84 GiB of expert weights. Host mirrors add 42 GiB beyond the primary CPU pool. HDD demand falls from 32.2% to zero. For a homogeneous pool, primary coverage is $L(K_H + K_C)s$. Physical host storage adds primary CPU weights, HBM mirrors, and staging. This accounts for the expert set and the copies that support demotion.

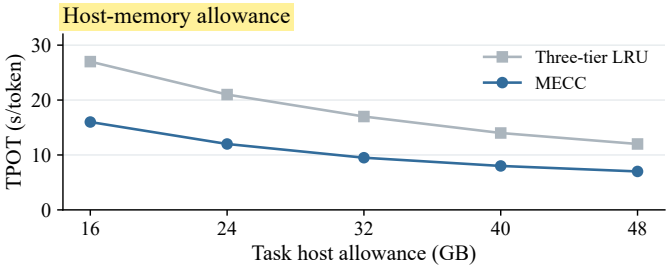


Fig. 14. Decode latency across host-memory allowances.

Figure 14 sweeps primary host allowances from 16 to 48 GiB. MECC TPOT changes from 16 to 7 s/token, versus 27 to 12 for three-tier LRU. Its first increment saves 4 s/token; the last saves 1. Earlier slots cover shorter reuse distances. E4 separately reaches complete expert coverage at 42 GiB HBM plus 42 GiB CPU residency.

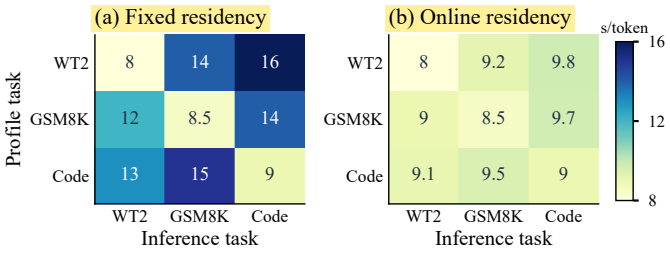


Fig. 15. Profile transfer across tasks. Rows select the calibration task and columns the inference task. Cells are TPOT in s/token; the two panels share a color scale.

Across Figure 15's six off-diagonal task pairs, fixed and online residency average 14.00 and 9.38 s/token, respectively. This gives a 33.0 percent reduction. Their diagonal means are both 8.50 s/token. Online admission adjusts the cached expert set to the serving task. Calibration size controls the initial profile.

F. Calibration Coverage and Input Length

Increasing calibration from 32 to 2048 samples changes TPOT from 10.8 to 8.0 s/token in Figure 13(a). The final expansion yields a smaller reduction than the first. Once the ranked hot set stabilizes, additional samples affect fewer initial placements.

Figure 13(b) varies input length from 32 to 2048 tokens. Endpoint first-token speedups are 1.61× and 1.79×. At the longer input, TTFT is 1360 seconds for MECC and 2440 for streaming. Prefill rows assigned to one expert share fetched weights and amortize delivery. Cache residency also reuses those weights across prompts.

G. Batch Throughput and Sequence Progress

Increasing batch size from one to eight raises MECC's aggregate rate from 0.125 to 0.505 tokens/s, or 4.04×, in Figure 13(c). For synchronous batches, the step interval is $B/R(B)$. These rates give 8.00 and 15.84 seconds at batches one and eight. The batch-step interval grows by 1.98 times as the batch grows by 8 times. Each step processes more rows and exposes more active expert work. Reporting both aggregate rate and step interval connects shared weight delivery to the progress of an individual sequence.

H. Amortizing Initial Placement

Let ΔI be the extra initialization time and ΔD the per-token decode saving. The E3 startup control crosses at $n^\star = \Delta I/\Delta D$. Figure 9(b) uses 18 seconds of extra initialization and 2 seconds of saving, giving 9 output tokens.

At 64 tokens, total average costs are 8.47 s/token for compiled initialization and 10.19 for online initialization. For requests $j = 1, \ldots, J$ sharing the cache, placement contributes $T_{\text{init}}/\sum_j n_j$ per output token. Here n_j is the output count of request j. Reusing the resident state pools these outputs into one amortization horizon. Reinitialization begins a new cache lifetime.

VII. Conclusion

MECC compiles initial residency and carries expert reuse through joint HBM/CPU retention. The plan specifies capacities and expert IDs; runtime requests drive promotion, demotion, and delivery. State reuse amortizes initial placement across requests. E1 gives 1.77× decode speedup over streaming on one A100.

References

[1] N. Shazeer, A. Mirhoseini, K. Maziarz, A. Davis, Q. Le, G. Hinton, and J. Dean, "Outrageously large neural networks: The sparsely-gated mixture-of-experts layer," in *International Conference on Learning Representations*, 2017.

[2] W. Fedus, B. Zoph, and N. Shazeer, "Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity," *Journal of Machine Learning Research*, vol. 23, no. 120, pp. 1–39, 2022.

[3] A. Q. Jiang, A. Sablayrolles, A. Roux *et al.*, "Mixtral of experts," *arXiv preprint arXiv:2401.04088*, 2024.

[4] D. Dai, C. Deng, C. Zhao *et al.*, "DeepSeekMoE: Towards ultimate expert specialization in mixture-of-experts language models," in *Proc. ACL*, 2024, pp. 1280–1297.

[5] S. Rajbhandari, C. Li, Z. Yao *et al.*, "DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale," in *Proc. ICML*, ser. Proceedings of Machine Learning Research, vol. 162, 2022, pp. 18 332–18 346.

[6] C. Hwang, W. Cui, Y. Xiong *et al.*, "Tutel: Adaptive mixture-of-experts at scale," in *Proc. MLSys*, 2023.

[7] O. J. Aimuyo, B. Oh, and R. Singh, "FlashMoE: Fast distributed MoE in a single kernel," *arXiv preprint arXiv:2506.04667v3*, 2025.

[8] A. Eliseev and D. Mazur, "Fast inference of mixture-of-experts language models with offloading," *arXiv preprint arXiv:2312.17238*, 2023.

[9] R. Kong, Y. Li, Q. Feng, W. Wang, X. Ye, Y. Ouyang, L. Kong, and Y. Liu, "SwapMoE: Serving off-the-shelf MoE-based large language models with tunable memory budget," *arXiv preprint arXiv:2308.15030*, 2023.

[10] L. Xue, Y. Fu, Z. Lu, L. Mai, and M. Marina, "MoE-Infinity: Efficient MoE inference on personal machines with sparsity-aware expert cache," *arXiv preprint arXiv:2401.14361v3*, 2025.

[11] X. Song, Z. Zhong, R. Chen, and H. Chen, "ProMoE: Fast MoE-based LLM serving using proactive caching," *arXiv preprint arXiv:2410.22134*, 2024.

[12] K. Kamahori, T. Tang, Y. Gu, K. Zhu, and B. Kasikci, "Fiddler: CPU-GPU orchestration for fast inference of mixture-of-experts models," in *International Conference on Learning Representations*, 2025.

[13] Y. Zhang, S. Aggarwal, and T. Mitra, "DAOP: Data-aware offloading and predictive pre-calculation for efficient MoE inference," *arXiv preprint arXiv:2501.10375*, 2025.

[14] S. Cao, S. Liu, T. Griggs, P. Schafhalter, X. Liu, Y. Sheng, J. E. Gonzalez, M. Zaharia, and I. Stoica, "MoE-Lightning: High-throughput MoE inference on memory-constrained GPUs," in *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, 2025.

[15] K. Yang, Y. Gao, X. Li *et al.*, "MoE-CORE: Coordinated expert offloading and residency for memory-constrained MoE inference," *arXiv preprint arXiv:2610.01950*, 2026.

[16] F. Liang, Y. Shen, J. Hu, J. Xu, H. Gao, M. Xu, and Z. Meng, "SSD-LLaMA: SSD-native inference for trillion-parameter MoE at 1+ token/s on a consumer PC," *arXiv preprint arXiv:2609.18110*, 2026.

[17] B. Kim, J. Lee, D. Han, H.-J. Yoo, and S. Kim, "FlashMoE: Reducing SSD I/O bottlenecks via ML-based cache replacement for mixture-of-experts inference on edge devices," *arXiv preprint arXiv:2601.17063v2*, 2026.

[18] D. Lepikhin, H. Lee, Y. Xu *et al.*, "GShard: Scaling giant models with conditional computation and automatic sharding," in *Proc. ICLR*, 2021.

[19] DeepSeek-AI, A. Liu, B. Feng *et al.*, "DeepSeek-V2: A strong, economical, and efficient mixture-of-experts language model," *arXiv preprint arXiv:2405.04434*, 2024.

[20] ——, "DeepSeek-V3 technical report," *arXiv preprint arXiv:2412.19437*, 2024.

[21] Mistral AI, "Cheaper, better, faster, stronger," Official Mixtral 8x22B release, 2024. [Online]. Available: https://mistral.ai/news/mixtral-8x22b

[22] T. Gale, D. Narayanan, C. Young, and M. Zaharia, "MegaBlocks: Efficient sparse training with mixture-of-experts," in *Proc. MLSys*, 2023.

[23] Z. Doucet, R. Sharma, M. de Vos, R. Pires, A.-M. Kermarrec, and O. Balmau, "HarMoEny: Efficient multi-GPU inference of MoE models," *arXiv preprint arXiv:2506.12417v2*, 2025.

[24] W. Kwon, Z. Li, S. Zhuang *et al.*, "Efficient memory management for large language model serving with PagedAttention," in *Proc. SOSP*, 2023.

[25] L. Zheng, L. Yin, Z. Xie *et al.*, "SGLang: Efficient execution of structured language model programs," in *Advances in Neural Information Processing Systems*, vol. 37, 2024.

[26] Y. Sheng, L. Zheng, B. Yuan *et al.*, "FlexGen: High-throughput generative inference of large language models with a single GPU," in *Proc. ICML*, ser. Proceedings of Machine Learning Research, vol. 202, 2023, pp. 31 094–31 116.

[27] R. Y. Aminabadi, S. Rajbhandari *et al.*, "DeepSpeed-Inference: Enabling efficient inference of transformer models at unprecedented scale," in *Proc. SC*, 2022.

[28] S. Rajbhandari, O. Ruwase, J. Rasley, S. Smith, and Y. He, "ZeRO-Infinity: Breaking the GPU memory wall for extreme scale deep learning," *arXiv preprint arXiv:2104.07857*, 2021.

[29] K. Alizadeh, S. I. Mirzadeh, D. Belenko *et al.*, "LLM in a flash: Efficient large language model inference with limited memory," in *Proc. ACL*, 2024, pp. 12 562–12 584.

[30] Y. Song, Z. Mi, H. Xie, and H. Chen, "PowerInfer: Fast large language model serving with a consumer-grade GPU," in *Proc. SOSP*, 2024.

[31] X. Zhao, B. Jia, H. Zhou, Z. Liu, S. Cheng, and Y. You, "HeteGen: Heterogeneous parallel inference for large language models on resource-constrained devices," *arXiv preprint arXiv:2403.01164*, 2024.

[32] P. Tang, J. Liu, X. Hou *et al.*, "HOBBIT: A mixed precision expert offloading system for fast MoE inference," *arXiv preprint arXiv:2411.01433v2*, 2024.

[33] W. Wang, X. Hou, X. Xia *et al.*, "Specter: Fast MoE inference via quantization-enabled multi-step speculative offloading," *arXiv preprint arXiv:2511.14102v3*, 2026.

[34] J. Yan, J. Liu, H. Xu, and L. Huang, "Accelerating mixture-of-expert inference with adaptive expert split mechanism," *arXiv preprint arXiv:2509.08342*, 2025.

[35] Y. Choi, S. Kim, J. Oh, G. Park, B. Kim, and H.-J. Yoo, "SliceMoE: Bit-sliced expert caching under miss-rate constraints for efficient MoE inference," *arXiv preprint arXiv:2512.12990*, 2025.

[36] S. Yadav and B. Asgari, "Mira: Memory-efficient MoE inference using adaptive caching and predictive expert staging," *arXiv preprint arXiv:2609.38090*, 2026.

[37] Y. Yang, Y. Zhao, P. Yang, S. Wang, and Z.-H. Zhou, "ZipMoE: Efficient on-device MoE serving via lossless compression and cache-affinity scheduling," in *Proceedings of the 43rd International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 306, 2026, pp. 147 261–147 284.

[38] J. Liang, S. Wang, M. Tian *et al.*, "Not all models suit expert offloading: On local routing consistency of mixture-of-expert models," *arXiv preprint arXiv:2505.16056v4*, 2026.

[39] R. Hwang, J. Wei, S. Cao, C. Hwang, X. Tang, T. Cao, and M. Yang, "Pre-gated MoE: An algorithm-system co-design for fast and scalable mixture-of-expert inference," in *Proc. ISCA*, 2024.

[40] Z. Fang, Z. Hong, Y. Huang *et al.*, "Fate: Fast edge inference of mixture-of-experts models via cross-layer gate," *arXiv preprint arXiv:2502.12224v2*, 2025.

[41] X. He, S. Zhang, K. Tang *et al.*, "ExpertFlow: Efficient mixture-of-experts inference via predictive expert caching and token scheduling," *arXiv preprint arXiv:2410.17954v2*, 2026.

[42] Z. Fang, Y. Huang, Z. Hong, Y. Lyu, W. Chen, Y. Yu, F. Yu, and Z. Zheng, "Klotski: Efficient mixture-of-expert inference via expert-aware multi-batch pipeline," in *Proc. ASPLOS*, 2025.

[43] H. Yu, I. Gim, K. Lee, and L. Zhong, "Accelerating offloading-based MoE inference by interleaving concurrent requests," *arXiv preprint arXiv:2611.04794*, 2026.

[44] L. Wang, Y. Du, Y. Pan, S. C. Liew, J. Liu, and K. Chen, "OD-MoE: On-demand expert loading for cacheless edge-distributed MoE inference," *arXiv preprint arXiv:2512.03927*, 2025.

Appendix A
Experimental Configuration and Figure Data

A. Platform and Physical Accounting

The A100 exposes 80 GB HBM. The host has about 1 TB of physical RAM. The backing volume is local ext4 on RAID device /dev/sdb, reporting ROTA=1. Physical-drive inventory and isolated disk bandwidth were not recorded. HDD names this backing path throughout the draft. Cold reads use the expert store; the medium label does not imply an SSD benchmark.

Each expert contains 352,321,536 bytes (336 MiB). The 256 experts occupy 84 GiB (90.19 decimal GB). Dense weights add to full-model size. The main plan holds 128 HBM experts (42 GiB) and 96 CPU experts (31.5 GiB). Host mirrors add 42 GiB. At E4's 60 GiB CPU-slot allowance, 128 CPU experts add 42 GiB of primary CPU residency; mirrors bring host weight storage to 84 GiB before staging. The allowance is a planner input for primary slots, not a physical host-memory cap. Legacy config fields named GB use GiB-sized slots.

B. Protocol Map and Data Identity

E1 and E4 use committed single A100 runs, one per policy and setting. E1 uses uniform initialization, eight WikiText-2 prompts at 64/64 input/output tokens, and 1024 tokens for PPL. E4 uses four prompts and 16 output tokens. Thus E1's 1.77× and E4's 1.42×/5.17× refer to distinct protocols.

Yellow-highlighted values and panel titles identify authored, unmeasured design targets. E2/E3/E5 and the calibration, quota, transfer, batch, and deployment scenarios are target arrays, not completed benchmark measurements. E2 uses GSM8K with 128/128 tokens, eight prompts, and calibrated initialization. Its model-specific H/C allowances are 30/20 GiB for Mixtral-8x7B, 30/16 for Qwen2-57B-A14B, and 50/64 for Mixtral-8x22B. E3 uses 50/40 for 8x7B and 50/64 for 8x22B, on GSM8K/HumanEval, with 2048 calibration samples. Its 24/8 example and 3.00–3.26× ratios do not describe E1. These workload assignments specify planned executions; the Code arrays map to HumanEval. All target-side configurations still require implementation and hardware checks. E3's I/O step specifies a scheduling target; the current decode path delivers experts on demand.

E5 is a matched FP16 policy-path design at 30/20 GiB on Mixtral-8x7B/GSM8K, eight 128/128-token prompts. The MoE-CORE, MoE-Infinity, SSD-LLaMA, and FlashMoE-RF entries are authored latency targets for local path adaptations. They are not executions of the authors' systems or observed improvements over them. The target ordering does not replace the E1 RF observation. Their raw targets, assumptions, and status are in BASELINE_TARGETS.json. Substitution, quantization, and compression are outside this comparison.

C. Complete E1 Results and Controls

E1 TPOT in seconds is 13.04259 for streaming, 21.75165 for SwapMoE-style, 8.25899 for Fiddler-style, 7.19497 for FlashMoE-RF, 7.78096 for MECC I/O, and 7.38198 for MECC. Independent caching gives 32.94189. Streaming/MECC TTFT is 156.85501/139.14026 seconds; Fiddler-style TTFT is 398.446. Full-precision division gives 4.46247× for independent/MECC TPOT and 2.86363× for Fiddler/MECC TTFT. Ratios in the figures are rounded independently. E1 FlashMoE-RF gives 1.81× decode and 1.54× first-token speedup over streaming; MECC gives 1.77×/1.13×.

Streaming retains the initialized GPU set. SwapMoE-style retains a fixed CPU set and uses GPU LRU. Fiddler-style computes non-HBM experts on CPU. RF uses equal-weight inverse-recency/frequency host scoring and GPU LRU; it does not contain the original trained predictor. These are local mechanism adapters. Main GPU PPL is 3.427; CPU-compute gives 3.432. The reported speedups have one run each and no estimated sampling variance.

D. Counters, Calibration, and Correctness

Main HDD shares aggregate classified expert accesses across warmup, prefill, decode, and PPL. MECC's counts are 29,558 HBM, 3,820 CPU, and 4,764 HDD, totaling 38,142. Prefetched experts are classified by residency at consumption. Physical reads and H2D have separate byte counters. Initialization reads are outside this demand denominator. Target stacks describe their specified protocols and sum to 100%.

The rank allocator consumes supplied skew parameters and count-ranked IDs. Profiling utilities regress positive log-counts on log-ranks; fit residuals for Mixtral are not available. E1 uses the uniform CLI branch; the non-uniform CLI generates skew unless an explicit compiler input supplies it. The plan is serialized configuration data, not generated GPU kernel code.

G1 compares the streaming and MECC paths through their shared dispatch on two 21/22-token fixture inputs, three greedy output tokens each, plus 128-token diagnostic PPL. Its per-case route/token equality is true; gate/logits maximum absolute errors are 0.0. This is short-window numerical equality between those paths, not an independent full-resident equivalence proof. No GSM8K or HumanEval task accuracy has been measured. G1_EVIDENCE.json records the source Git object and fixture hashes.

E. Arithmetic and Deployment Scenario

The transfer matrix's six fixed off-diagonal cells sum to 84, giving 14.00. Adaptive cells sum to 56.3, giving 9.3833 and a 32.976% reduction. The startup control uses 12/30-second initialization and 10/8-second decode. Its crossing is 9 tokens; 64 tokens is an amortization example. The resident-reference scenario in Figure 16 uses authored single/multiple-GPU values.

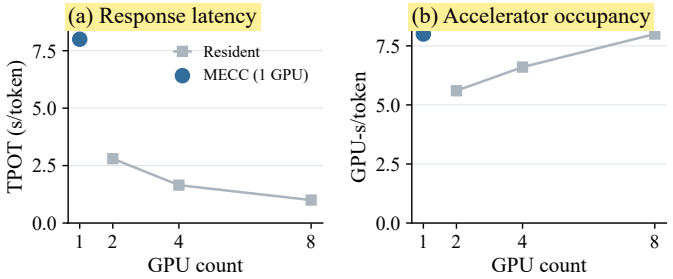


Fig. 16. Deployment scenario: latency and accelerator occupancy for one-GPU MECC and resident references. Yellow titles follow the target-array convention above.